

Operating Systems (OS) — Assignment

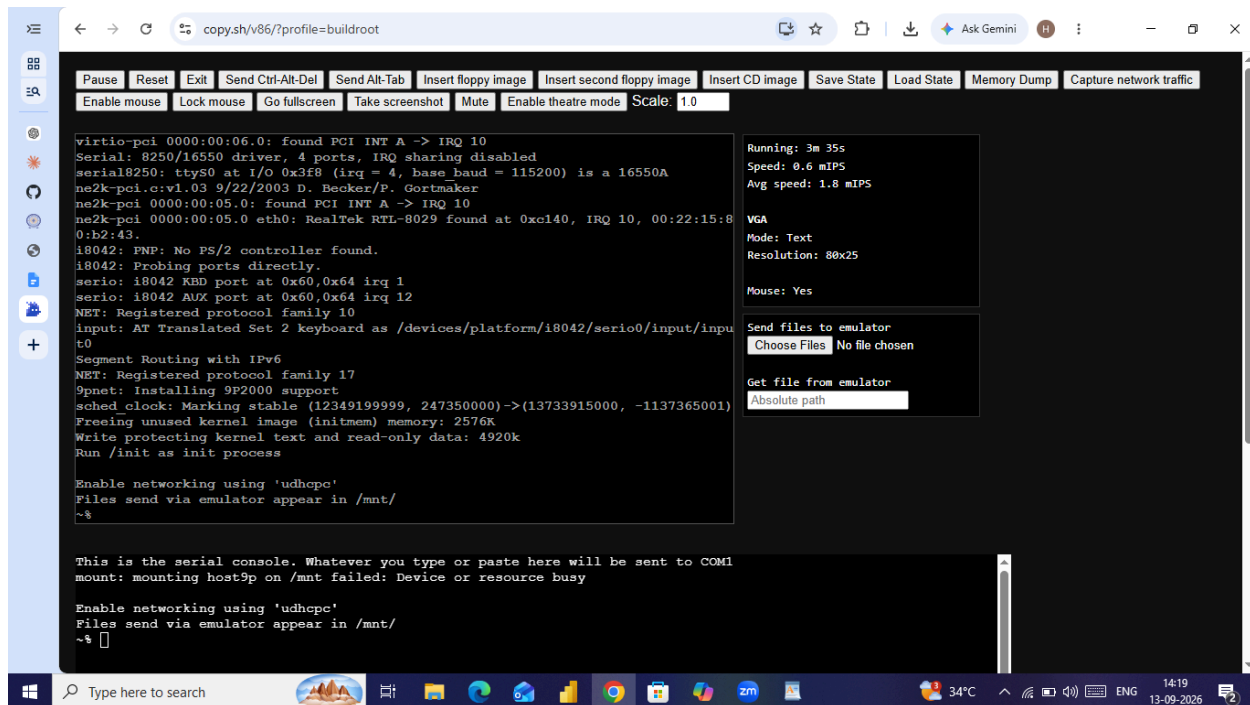
NAME : HARDIK KUMAR

ROLL NO. : B24BS2119

DATE : 13 SEPTEMBER 2026

Task 1 — Exploring a Virtual Operating System

Open a virtual operating system using copy.sh and explore its interface.



a. Name of the operating system explored

→ Buildroot (a minimal embedded Linux distribution), run via the v86 emulator on copy.sh.

b. Four features or components observed

Boot messages – text shown when the system starts, listing hardware like keyboard, network card, etc.

Terminal (console) – a black screen with a prompt (~\$) where you type commands

Networking option – it can connect to a network using a tool called **udhcpd**

Emulator control buttons – buttons like Pause, Reset, Save State, Take Screenshot (these belong to the v86 emulator, not the OS itself)

c. Which part of the interface allows you to interact with files and folders?

This system has no graphical file manager (no clickable folders/icons). You interact with files only through the **terminal**, by typing commands like `ls` (list files) or `cd` (change folder)

d. Which part can be used to execute commands?

→ terminal/console

Task 2 — Comparing the Virtual OS with Google Colab

Feature	Virtual OS (Buildroot)	Google Colab
Graphical desktop available	No — only a text terminal, no desktop	No — only a notebook (code cells), no desktop either
Can run commands	Yes — via the terminal	Yes — using <code>!command</code> in a code cell
Can create files	Yes — using terminal commands	Yes — through code or by uploading
Can run Python programs	Only if Python is installed in Buildroot (not by default)	Yes — Python is already installed and ready to use
User directly controls the complete OS	Yes — full access to the emulated machine	No — Colab runs in a temporary, limited cloud VM

a. Two similarities:

- Both let you run commands/programs.
- Both let you create and work with files.

b. Two differences:

- Colab is Python/notebook-focused with ready-to-use tools; Buildroot is a bare minimal Linux system with almost nothing pre-installed.
- Buildroot gives full control over the whole machine; Colab only gives limited, temporary access to a cloud VM.

c. Why doesn't Colab behave like a normal computer's OS?

Because Colab doesn't give you real control of an operating system — it just gives you a notebook connected to a temporary virtual machine in the cloud. There's no desktop, no permanent storage, and no direct hardware access like a real/full OS would have.

Task 3 — Programs vs Processes

Suppose the following applications are running on a computer: Web browser, Music player, Text editor, Calculator.

a. Which of these applications can become a process when executed?

All four — the Web browser, Music player, Text editor, and Calculator — can become a process when executed. Any program that is loaded into memory and run by the operating system becomes a process.

b. What is the difference between a program and a process?

A program is a passive set of instructions stored on disk (like an executable file) that does nothing on its own. A process is an active instance of a program that is currently being executed by the CPU, with its own memory space and execution state. In short, a program is code sitting in storage, while a process is that code actually running.

c. If you open the browser three times, can the operating system manage multiple processes?

Yes. Each time the browser is opened, the operating system creates a separate process for it, with its own process ID (PID), memory space, and resources. The OS is designed to manage many processes at the same time, whether they come from the same program or different programs.

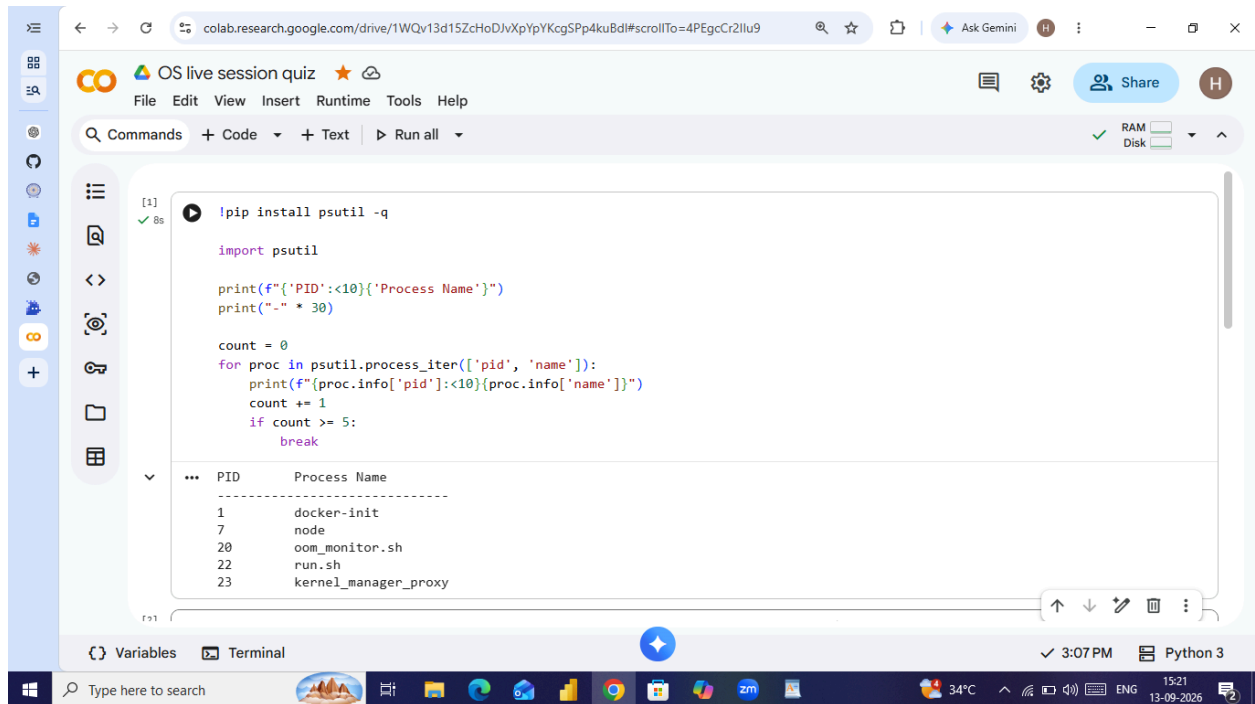
d. Why does the operating system need to keep track of running processes?

The operating system needs to track every running process so it can share CPU time fairly between them, manage memory so processes don't interfere with each other, allow communication between processes when needed, detect and handle errors or crashes, and properly close processes and free their resources once they finish.

Task 4 — Monitoring Processes with psutil

In Google Colab, install/import the psutil library and use it to view running processes.

Python code:



```
[1] ✓ 8s
!pip install psutil -q

import psutil

print(f"{'PID':<10}{'Process Name'}")
print("-" * 30)

count = 0
for proc in psutil.process_iter(['pid', 'name']):
    print(f"{proc.info['pid']:<10}{proc.info['name']}")
    count += 1
    if count >= 5:
        break
```

PID	Process Name
1	docker-init
7	node
20	oom_monitor.sh
22	run.sh
23	kernel_manager_proxy

a. What does PID represent?

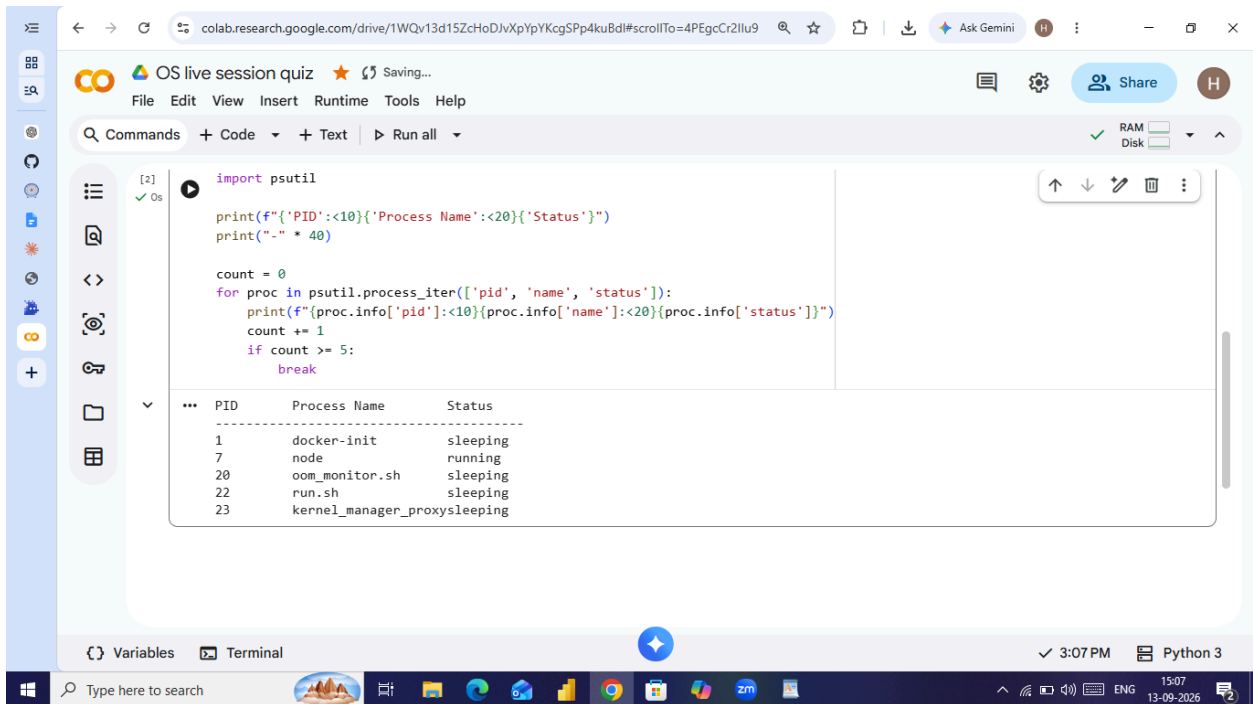
PID stands for Process ID — a unique number that the operating system assigns to every running process. It allows the OS to identify and distinguish one running process from another, even if two processes are running the same program.

b. Why should every running process have an identifier?

Every process needs an identifier so the operating system can uniquely track and manage it — for example, to give it CPU time and memory, to stop or restart a specific process, and to tell apart multiple copies of the same program running at once. Without a unique ID, the OS wouldn't be able to control processes individually.

Task 5 — Adding Process Status

Modify the process-monitoring code so it also displays process status.



```
import psutil

print(f"{'PID':<10}{'Process Name':<20}{'Status'}")
print("-" * 40)

count = 0
for proc in psutil.process_iter(['pid', 'name', 'status']):
    print(f"{proc.info['pid']:<10}{proc.info['name']:<20}{proc.info['status']}")
    count += 1
    if count >= 5:
        break
```

PID	Process Name	Status
1	docker-init	sleeping
7	node	running
20	oom_monitor.sh	sleeping
22	run.sh	sleeping
23	kernel_manager_proxysleeping	sleeping

a. Did all processes have the same status?

No, not all processes had the same status. Most processes showed "sleeping" while one (the currently active one) showed "running", because at any moment only one process can actively use a CPU core while the rest wait. (Apni actual table dekh kar isse confirm/adjust kar lena.)

b. What does the process status tell us?

The status tells us the current state of a process — whether it is actively using the CPU (running), waiting for its turn (ready), waiting for some event or resource like input or disk access (sleeping/waiting), or has finished (terminated). It shows what the process is doing right now.

c. Why might the status of a process change while the system is running?

A process's status changes as it interacts with the CPU and other resources over time. For example, a running process may need to wait for input or a file to load, so its status changes to waiting; once that resource is ready, it becomes ready again; and once the CPU picks it, it becomes running again. This keeps happening as the OS shares limited CPU time among many processes.

Task 6 — Process States

Simplified process states: Running (currently using the CPU), Ready (waiting to get CPU time), Waiting/Blocked (waiting for an event or resource), Terminated (execution has completed). Identify the most suitable process state for each situation:

For each situation below, identify the most suitable process state:

- a. A program is currently performing calculations on the CPU. → **Running**
- b. A program is ready to run but another process is currently using the CPU. → **Ready**
- c. A program is waiting for the user to provide input. → **Waiting/Blocked**
- d. A program has completed execution. → **Terminated**
- e. A program is waiting for data to be read from a file. → **Waiting/Blocked**

Task 7 — Process State Diagram

Draw a simple diagram showing the movement of a process between the following states:

Ready → Running → Waiting → Ready → Running → Terminated

[Ready] → [Running] → [Waiting] → [Ready] → [Running] → [Terminated]

Note: This linear diagram shows one possible path a process takes. In a fuller state diagram this would be drawn as a cycle: Ready ⇌ Running, Running → Waiting → Ready, and Running → Terminated, since a process can repeat the Ready↔Running↔Waiting loop many times before finally terminating.

a. What causes a process to move from Ready to Running?

The CPU scheduler selects the process from the ready queue (based on a scheduling algorithm and its priority) and assigns it the CPU, moving it from Ready to Running.

b. Why can a running process move into the Waiting state?

A running process moves into the Waiting state when it requires an event or resource that isn't immediately available — such as I/O completion, user input, or a lock held by another process — so it cannot continue executing until that condition is satisfied.

c. After the required resource becomes available, which state can the process move to?

It moves to the Ready state, where it waits to be scheduled onto the CPU again (it does not go directly back to Running).

d. Which component of the operating system helps decide which ready process gets CPU time?

The CPU Scheduler (a part of the process management component of the OS) decides which ready process is given CPU time next, based on a scheduling algorithm such as First-Come-First-Served, Round Robin, Shortest Job First, or Priority Scheduling.

Task 8 — Multitasking and Resource Management

Scenario: A student is simultaneously using a browser with multiple tabs, a video player, a Python program, a file download, and a text editor, on a computer with limited CPU and memory.

Not all of these applications actually use system resources at the exact same instant, even though they appear to run simultaneously. On a machine with a limited number of CPU cores, the operating system rapidly switches the CPU between processes — giving each one a small time slice — so it only looks like everything is happening at once; this is called multitasking through time-sharing. The operating system manages these running processes by scheduling CPU time fairly among them, allocating and protecting separate memory space for each one, and handling background tasks like the file download largely through I/O operations that don't need constant CPU attention. If one process, such as the video player or a runaway Python script, consumes too much CPU or memory, other processes can slow down, become unresponsive, or in severe cases the whole system may freeze or crash due to resource exhaustion. This is exactly why process monitoring is useful: by observing CPU and memory usage per process, a user or administrator can identify which application is causing a slowdown and take action, such as closing or restarting it. If the file download is still in progress, that specific process would likely be in the Waiting/Blocked state, since it is waiting on network I/O rather than actively using the CPU. Ultimately, process management is a core responsibility of an operating system because it ensures fair and efficient sharing of limited hardware resources among many competing programs, keeps the system stable and responsive, and prevents any single misbehaving process from disrupting the entire system.